Course Code: INT 315

Course Title: Cluster Computing

Continuous Assessment: 02

Date Of Submission: 09-April-2026

Topic: Water Quality Classification using Decision Tree

Name: Debasmita Chatterjee

Registration No.: 12219423

Roll No.: REM301A63

Section: EM301

TABLE OF CONTENT

TITLE 1
ABSTRACT 1
INTRODUCTION 1
PROBLEM STATEMENT 1
OBJECTIVES 1
DATASET DESCRIPTION 2
DATA PREPROCESSING 2
MODEL BUILDING 3
EVALUATION METRICS 4
RESULTS 4
VISUALIZATION 4
DISCUSSION 5
CONCLUSION 5
FUTURE WORK 5
IMPLEMENTATION 5
LINKS 12

TITLE

Water Quality Classification using Decision Tree (PySpark)

ABSTRACT

This project focuses on classifying water samples as safe or unsafe for human consumption using chemical and physical water quality parameters. A Decision Tree classifier was implemented using Apache Spark to efficiently process the dataset. Missing values were handled using median imputation, and class imbalance was reduced through upsampling. The model was trained using cross-validation for better performance and achieved an accuracy of 72.57%. Visualization techniques such as confusion matrix heatmap and feature importance bar chart were used to analyze model performance and identify the most influential water contaminants.

INTRODUCTION

Water quality assessment is essential for public health and environmental sustainability. Contaminated water can cause severe health issues, making water safety prediction an important problem.

Machine learning techniques can help classify water samples based on physical and chemical characteristics. This project uses a Decision Tree classifier in PySpark to predict whether water is potable or not.

PROBLEM STATEMENT

The objective of this project is to classify water samples as safe or unsafe for drinking based on water quality attributes such as pH, hardness, solids, chloramines, sulfate, conductivity, and turbidity.

The challenge lies in handling missing values, class imbalance, and identifying the most important contaminants affecting water quality.

OBJECTIVES

- To load and preprocess the water quality dataset using PySpark
- To handle missing values effectively
- To balance the dataset using upsampling
- To perform feature engineering

- To build a Decision Tree classification model
- To evaluate model performance using accuracy and F1-score
- To visualize confusion matrix and feature importance

DATASET DESCRIPTION

The dataset contains water quality parameters and a target variable indicating water potability.

Key features used:

- pH
- Hardness
- Solids
- Chloramines
- Sulfate
- Conductivity
- Organic Carbon
- Trihalomethanes
- Turbidity

Target:

- Potability (Safe / Unsafe)

The dataset consists of 3276 water samples.

DATA PREPROCESSING

Data preprocessing was performed to improve model quality.

1. Handling Missing Values
   Missing values in pH, Sulfate, and Trihalomethanes were replaced using median values.
2. Label Preparation
   The Potability column was renamed to label for classification.
3. Class Balancing
   The minority class was upsampled to reduce dataset imbalance.

4. Feature Assembly

   Selected features were combined into a single feature vector using VectorAssembler.

FEATURE ENGINEERING

The following water quality parameters were selected as input features:

- pH
- Hardness
- Solids
- Chloramines
- Sulfate
- Conductivity
- Organic Carbon
- Trihalomethanes
- Turbidity

These parameters significantly influence water potability and help the model learn classification patterns.

MODEL BUILDING

A Decision Tree classifier was implemented using PySpark MLlib.

Steps followed:

- Feature vector creation
- Train-test split (80:20)
- Hyperparameter tuning using CrossValidator
- Model training on training dataset
- Predictions on test dataset

Cross-validation helped improve model generalization and reduce overfitting.

EVALUATION METRICS

The following metrics were used:

1. Accuracy

Measures overall classification correctness.

2. Confusion Matrix

Shows true positives, true negatives, false positives, and false negatives.

3. F1-Score

Measures balanced classification performance.

These metrics provide a complete view of model effectiveness.

RESULTS

The model achieved:

- Accuracy ≈ 72.57%
- F1-score ≈ 0.724

The model showed balanced performance for both safe and unsafe water samples.

This indicates that the Decision Tree classifier can effectively predict water potability.

VISUALIZATION

1. Confusion Matrix Heatmap

The confusion matrix heatmap was used to visualize model predictions and classification errors.

2. Feature Importance Bar Chart

Feature importance was used to identify the most influential water quality parameters.

Key important features included:

- Chloramines
- pH
- Sulfate

These factors had the highest impact on prediction.

DISCUSSION

The Decision Tree model performed well in classifying water quality, achieving realistic accuracy.

The confusion matrix showed balanced prediction performance, though some overlap between safe and unsafe classes still existed.

Feature importance analysis revealed that chemical disinfectants and acidity levels significantly influence water safety

CONCLUSION

This project successfully implemented a water quality classification system using Apache Spark.

The model achieved good performance with an accuracy of 72.57%, and feature importance analysis identified key contaminants affecting water safety.

The project demonstrates how machine learning can support water quality monitoring and public health decisions.

FUTURE WORK

Future improvements may include:

- Using ensemble models such as Random Forest
- Applying advanced feature engineering
- Increasing dataset size
- Integrating IoT sensor data for real-time prediction

IMPLEMENTATION

Step 1: Starting Spark Session

```
C:\Users\Diya>pyspark
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on w
in32
Type "help", "copyright", "credits" or "license" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
26/04/09 13:57:22 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java
.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apa
che.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/09 13:57:23 WARN NativeCodeLoader: Unable to load native-hadoop library for your platf
orm... using builtin-java classes where applicable
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 4.1.1
      /_/

Using Python version 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026 16:04:56)
Spark context Web UI available at http://DBMS:4040
Spark context available as 'sc' (master = local[*], app id = local-1775723244447).
SparkSession available as 'spark'.
```

Output: Spark Session Started

Step 2: Loading Dataset and Handling Missing Values

```
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 4.1.1
      /_/

Using Python version 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026 16:04:56)
Spark context Web UI available at http://DBMS:4040
Spark context available as 'sc' (master = local[*], app id = local-1775723244447).
SparkSession available as 'spark'.
>>> df = spark.read.csv("C:/Users/Diya/Downloads/water_potability.csv",heade\r=True,inferSch
ema=True)
>>> cols = ["ph", "Sulfate", "Trihalomethanes"]
>>> for c in cols:
...     median = df.approxQuantile(c, [0.5], 0.01)[0]
...     df = df.fillna({c: median})
...
>>> df = df.withColumnRenamed("Potability", "label")
>>> df.show(5)
+-----------------+------------------+------------------+-----------------+-----------------
-+------------------+------------------+------------------+------------------+-----+
|               ph|          Hardness|            Solids|      Chloramines|           Sulfat
e|      Conductivity|    Organic_carbon|   Trihalomethanes|         Turbidity|label|
+-----------------+------------------+------------------+-----------------+-----------------
-+------------------+------------------+------------------+------------------+-----+
|7.027185659271876| 204.8904554713363|20791.318980747026|7.300211873184757|368.5164413498033
6| 564.3086541722439|  10.3797830780847|  86.9909704615088|2.9631353806316407|    0|
| 3.71608007538699|129.42292051494425|18630.057857970347|   6.635245883862|332.8775736543078
7| 592.8853591348523|15.180013116357259| 56.32907628451764| 4.500656274942408|    0|
|8.099124189298397|224.23625939355776|19909.541732292393|9.275883602694089|332.8775736543078
7| 418.6062130644815|16.868636929550973| 66.42009251176368|3.0559337496641685|    0|
|8.316765884214679|214.37339408562252|22018.417440775294| 8.05933237743854|356.8861356430566
6| 363.2665161642437|18.436524495493302|100.34167436508008| 4.628770536837084|    0|
|9.092223456290965|181.10150923612525| 17978.98633892625|6.546599974207941|310.1357375242044
4|398.41081338184466|11.558279443446395|31.997992727424737| 4.075075425430034|    0|
+-----------------+------------------+------------------+-----------------+-----------------
-+------------------+------------------+------------------+------------------+-----+
only showing top 5 rows
>>>
```

Output: Dataset loaded successfully, missing values handled, and target column prepared for classification.

Step 3: Class Balance Check

```
>>> df.groupBy("label").count().show()
+-----+-----+
|label|count|
+-----+-----+
|    1| 1278|
|    0| 1998|
+-----+-----+
```

Output: Displayed class distribution, showing imbalance between safe and unsafe water samples.

Step 4: Balanced Dataset

```
>>> major_df = df.filter("label = 0")
>>> minor_df = df.filter("label = 1")
>>> minor_upsampled = minor_df.sample(withReplacement=True,fraction=1.5,seed=42)
>>> df = major_df.union(minor_upsampled)
>>> df.groupBy("label").count().show()
+-----+-----+
|label|count|
+-----+-----+
|    0| 1998|
|    1| 1933|
+-----+-----+
```

Output: Balanced the dataset by upsampling the minority class to reduce class imbalance.

Step 5: Feature Engineering

```
>>> from pyspark.ml.feature import VectorAssembler
>>> feature_cols = [
...     "ph",
...     "Hardness",
...     "Solids",
...     "Chloramines",
...     "Sulfate",
...     "Conductivity",
...     "Organic_carbon",
...     "Trihalomethanes",
...     "Turbidity"
... ]
>>> assembler = VectorAssembler(inputCols=feature_cols,outputCol="features")
>>> data = assembler.transform(df).select("features", "label")
>>> train, test = data.randomSplit([0.8, 0.2], seed=42)
>>>
```

Output: Features were combined into a single vector, and the dataset was split into training and testing sets.

Step 6: Train Decision Tree + Hyperparameter Tuning

```
>>> from pyspark.ml.classification import DecisionTreeClassifier
>>> from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
>>> from pyspark.ml.evaluation import MulticlassClassificationEvaluator
>>> dt = DecisionTreeClassifier>>>
>>> paramGrid = ParamGridBuilde
>>> from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
>>> from pyspark.ml.evaluation import MulticlassClassificationEvaluator
>>> dt = DecisionTreeClassifier(featuresCol="features",labelCol="label")
>>> paramGrid = ParamGridBuilder().addGrid(dt.maxDepth, [5, 10, 15]).addGrid(dt.maxBins, [16, 32, 64]).build()
>>> evaluator = MulticlassClassificationEvaluator(labelCol="label",metricName="accuracy")
>>> cv = CrossValidator(estimator=dt,estimatorParamMaps=paramGrid,evaluator=evaluator,numFolds=3)
>>> model = cv.fit(train)
>>> predictions = model.transform(test)
>>>
```

Output: Trained the Decision Tree model using cross-validation and generated predictions on the test dataset.

Step 7: Accuracy

```
>>> accuracy = evaluator.evaluate(predictions)
>>> print("Model Accuracy:", round(accuracy * 100, 2), "%")
Model Accuracy: 72.57 %
>>>
```

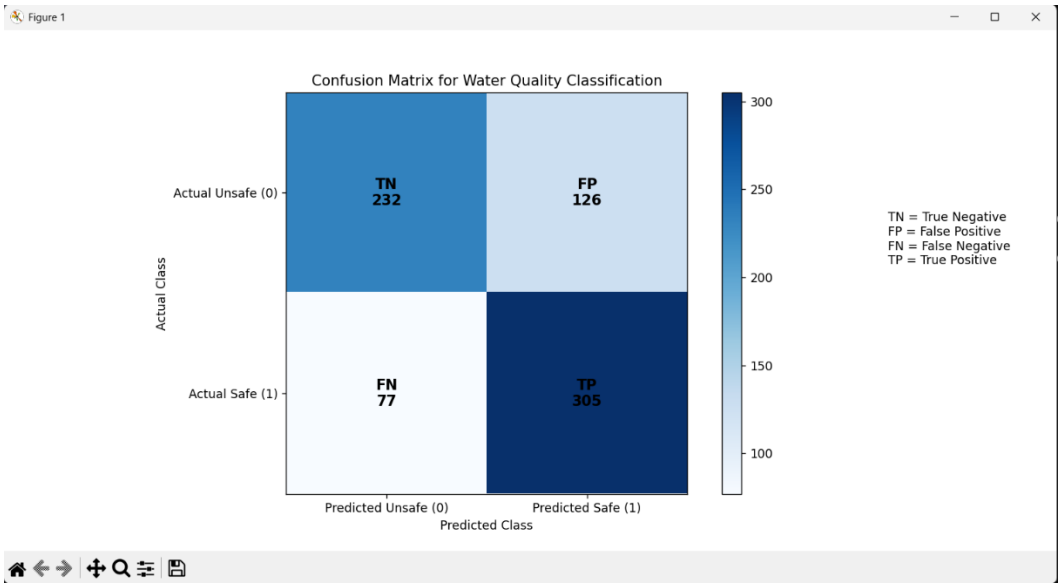
Output: The Decision Tree model achieved an accuracy of 72.57% on the test dataset.

Step 8: Confusion Matrix

```
>>> cm_df = predictions.groupBy("label", "prediction").count().toPandas()
>>> cm_matrix = cm_df.pivot(
...     index="label",
...     columns="prediction",
...     values="count"
... ).fillna(0)
>>> print(cm_matrix)
prediction  0.0  1.0
label
0           232  126
1            77  305
>>> |
```

Output: Generated the confusion matrix, showing correct and incorrect classifications for both safe and unsafe water samples.

Step 9: Confusion Matrix Heatmap




```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> cm = cm_matrix.values
>>>
... labels = np.array([
...     [f"TN\n{int(cm[0][0])}", f"FP\n{int(cm[0][1])}"],
...     [f"FN\n{int(cm[1][0])}", f"TP\n{int(cm[1][1])}"]
... ])
>>> plt.figure(figsize=(8, 6))
<Figure size 800x600 with 0 Axes>
>>> plt.imshow(cm, cmap="Blues")
<matplotlib.image.AxesImage object at 0x000001BBD69D4440>
>>> plt.colorbar()
<matplotlib.colorbar.Colorbar object at 0x000001BBD69D5940>
>>> plt.xticks([0, 1],["Predicted Unsafe (0)", "Predicted Safe (1)"])
([<matplotlib.axis.XTick object at 0x000001BBD5E3AE40>, <matplotlib.axis.XTick object at 0x
000001BBD69D9A90>], [Text(0, 0, 'Predicted Unsafe (0)'), Text(1, 0, 'Predicted Safe (1)')])
>>> plt.yticks([0, 1],["Actual Unsafe (0)", "Actual Safe (1)"])
([<matplotlib.axis.YTick object at 0x000001BBD5E3B620>, <matplotlib.axis.YTick object at 0x
000001BBD6999D10>], [Text(0, 0, 'Actual Unsafe (0)'), Text(0, 1, 'Actual Safe (1)')])
>>> for i in range(2):
...     for j in range(2):
...         plt.text(j,
...             i,
...             labels[i][j],
...             ha="center",
...             va="center",
...             fontsize=12,
...             fontweight="bold")
...
Text(0, 0, 'TN\n232')
Text(1, 0, 'FP\n126')
Text(0, 1, 'FN\n77')
Text(1, 1, 'TP\n305')
>>> plt.xlabel("Predicted Class")
Text(0.5, 0, 'Predicted Class')
>>> plt.ylabel("Actual Class")
Text(0, 0.5, 'Actual Class')
>>> plt.title("Confusion Matrix for Water Quality Classification")
Text(0.5, 1.0, 'Confusion Matrix for Water Quality Classification')
>>> plt.figtext(  0.84,
...     0.55,
...     "TN = True Negative\nFP = False Positive\nFN = False Negative\nTP = True Positive",
...     fontsize=10)
Text(0.84, 0.55, 'TN = True Negative\nFP = False Positive\nFN = False Negative\nTP = True P
ositive')
>>> plt.subplots_adjust(right=0.78)
>>> plt.show()
```

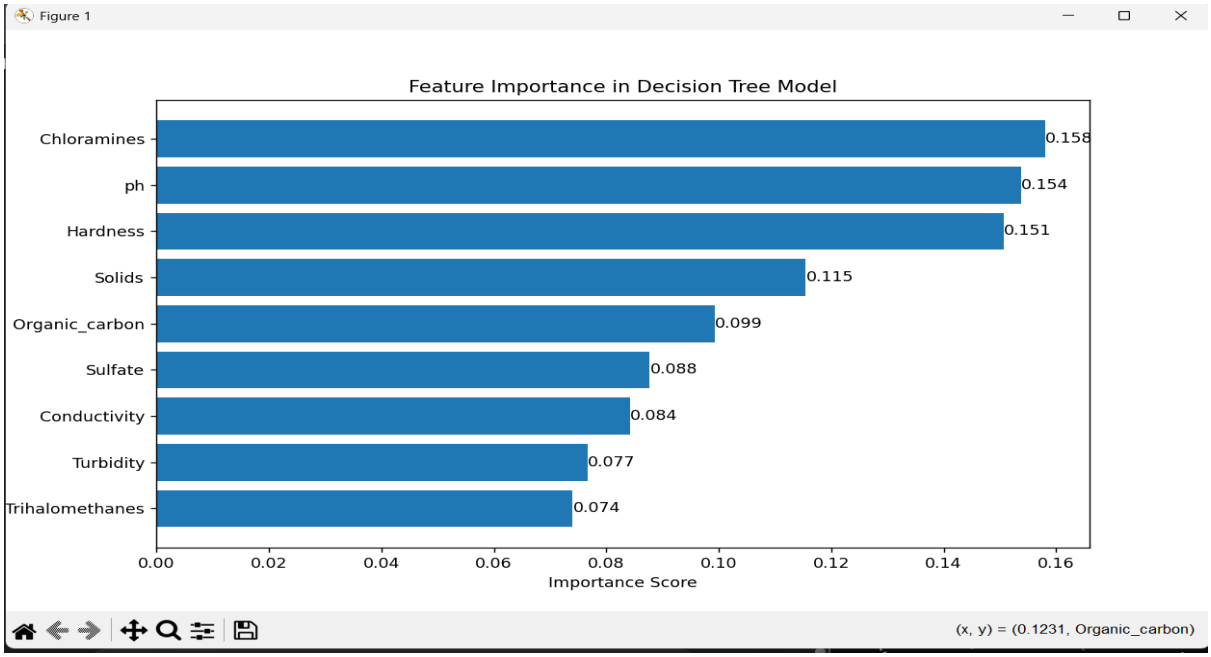
Output: Visualized the confusion matrix as a heatmap to clearly show true positives, true negatives, false positives, and false negatives.

Step 10: Feature Importance

```
>>> importance = model.bestModel.featureImportances.toArray()
>>> import pandas as pd
>>> df_imp = pd.DataFrame({
...     "Feature": feature_cols,
...     "Importance": importance
... }).sort_values(by="Importance", ascending=True)
>>> |
```

Output: Visualized feature importance to identify the most influential water quality parameters affecting classification.

Step 11: Feature Importance Graph




```
>>> plt.figure(figsize=(10, 6))
<Figure size 1000x600 with 0 Axes>
>>> plt.barh(df_imp["Feature"], df_imp["Importance"])
<BarContainer object of 9 artists>
>>> for i, v in enumerate(df_imp["Importance"]):
...     plt.text(
...         v,
...         i,
...         f"{v:.3f}",
...         va="center"
...     )
...
Text(0.07402296655147642, 0, '0.074')
Text(0.0767503484612956, 1, '0.077')
Text(0.0841795453065872, 2, '0.084')
Text(0.08774166107414448, 3, '0.088')
Text(0.09928971100680045, 4, '0.099')
Text(0.11548802026431218, 5, '0.115')
Text(0.15071443949158508, 6, '0.151')
Text(0.15376988113421305, 7, '0.154')
Text(0.1580434267095855, 8, '0.158')
>>> plt.xlabel("Importance Score")
Text(0.5, 0, 'Importance Score')
>>> plt.ylabel("Water Quality Parameters")
Text(0, 0.5, 'Water Quality Parameters')
>>> plt.title("Feature Importance in Decision Tree Model")
Text(0.5, 1.0, 'Feature Importance in Decision Tree Model')
>>> plt.show()
>>>
```

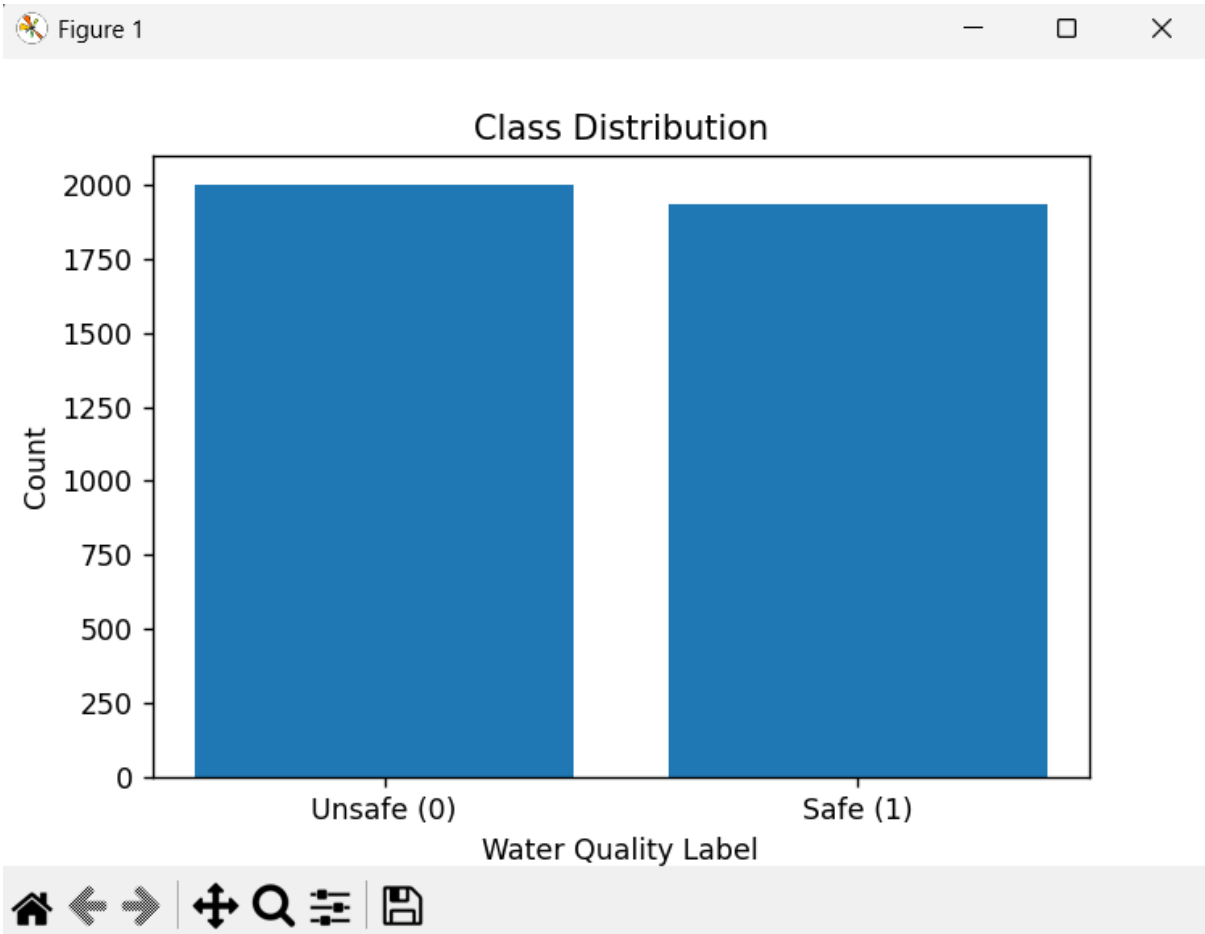
Output: Displayed the contribution of each water quality parameter to the Decision Tree model's prediction.

Step 12: F1-Score Evaluation

```
>>> for metric in ["weightedPrecision", "weightedRecall", "f1"]:
...     evaluator = MulticlassClassificationEvaluator(
...         labelCol="label",
...         predictionCol="prediction",
...         metricName=metric
...     )
...
>>> print(metric, ":", evaluator.evaluate(predictions))
f1 : 0.7238661411290964
>>>
```

Output: Evaluated model performance using F1-score, showing balanced classification performance with a score of 0.72.

Step 16: Class Distribution



Output: Visualized the distribution of safe and unsafe water samples in the dataset.

LINKS

GITHUB LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/tree/main

WORD LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Report.docx

PDF LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Report.pdf

PPT LINK: https://github.com/Aditya-227/Renewable-Energy-Output-Prediction/blob/main/Renewable-Energy-Output-Prediction.pptx